```
StatusService statusService;
...
private void updateStatusLevel() {
    setStatus(Status.statusLevelFor(statusPoints));
}
```

You've also discovered that the `FrequentFlyer` class needs the `StatusService` to do its job, so you'll need to make sure the `FrequentFlyer` class has an instance of `StatusService` available to use. There are many ways you could do this, such as using dependency injection frameworks like Spring or Guice in Java. But for testing purposes, you need to be able to provide your own custom version of the `StatusService` class:

```
protected FrequentFlyer(String frequentFlyerNumber,
                        String firstName,
                        String lastName,
                        Status status,
                        int statusPoints,
                        StatusService statusService) {
    ...
    this.statusService = statusService;
}
```

Pass a StatusService object in the constructor.

SPECIFY FOCUSED BEHAVIOR USING MOCK SERVICES

When you use this approach, a unit test focuses on specifying the behavior of a particular class or method and describes what it expects of any other classes it needs. This makes it very clear what other services your class needs to work. More interestingly, it gives you a chance to define the "contract" between your class and the service class you're calling.

Mocking libraries exist for all modern testing frameworks, and you can also write your own stubs and mocks if you prefer. The following code uses the popular Mockito library for Java (https://code.google.com/p/mockito/):

```
@Test
public void should_cumulate_points_with_each_flight() {

    // GIVEN
    StatusService statusService = mock(StatusService.class);
    when(statusService.statusLevelFor(300)).thenReturn(Status.Silver)

    FrequentFlyer member = new FrequentFlyer("12345678","Joe", "Bloggs",
                                             statusService);

    // WHEN
    member.earns(100).statusPoints();
    member.earns(200).statusPoints();

    // THEN
    assertThat(member.getStatusPoints(),is(greaterThanOrEqualTo((300))));
    assertThat(member.getStatus(), is(Status.Silver));
}
```

Use a mock version of the status service.

Return "Silver" for a value of 300.

Use the mock status service for this Frequent Flyer member.

Here you assume that the status service works, and that it returns the correct status when you ask about a given number of points. So this test could read, "Assuming that the status service tells us that the status level for 300 points is Silver, when a member